TopSecret - UML Class Diagram (Homework 5)
(A) access levels (B) user interface (C) model (D) accounts and audit log

Model (C)

Facility
- id: int
- name: String
- abbreviation: String

+ Facility(int, String, String)
+ getId(): int
+ getName(): String
+ getAbbreviation(): String
+ setName(String): void
+ setAbbreviation(String): void

ModelDatabase
- connection: Connection

+ ModelDatabase(Connection)
+ createTables(): void
- execute(String, Object...): int
- getLastInsertId(): int
+ addFacility(String, String): int
+ addAgent(String, String, String, String, int): int
+ addBrief(String, String, String): int
+ getFacilities(): List<Facility>
+ getAgents(): List<Agent>
+ getBriefs(): List<Brief>
+ assignMission(int, int, List<Integer>): void
+ updateBrief(int, String, String, String): boolean
+ updateAgent(int, String, String, String, String, int): boolean
+ updateFacility(int, String, String): boolean
+ deleteBrief(int): boolean
+ deleteAgent(int): boolean
+ deleteFacility(int): boolean
- deleteOrExplain(String, int, String): boolean
+ importSampleData(Path): int
~ readTsv(Path, int): List<String[]> {static}

Agent
- id: int
- name: String
- dateOfBirth: String
- dateOfDeath: String
- notes: String
- facilityId: int

+ Agent(int, String, String, String, String, int)
+ getId(): int
+ getName(): String
+ getDateOfBirth(): String
+ getDateOfDeath(): String
+ getNotes(): String
+ getFacilityId(): int
+ setName(String): void
+ setDateOfBirth(String): void
+ setDateOfDeath(String): void
+ setNotes(String): void
+ setFacilityId(int): void

Mission
- id: int
- title: String
- brief: String
- date: String
- facilityId: int
- agentIds: List<Integer>

+ Mission(int, String, String, String)
+ Mission(int, String, String, String, int, List<Integer>)
+ getId(): int
+ getTitle(): String
+ getBrief(): String
+ getDate(): String
+ getFacilityId(): int
+ getAgentIds(): List<Integer>
+ equals(Object): boolean
+ hashCode(): int

Brief
- id: int
- title: String
- date: String
- text: String

+ Brief(int, String, String, String)
+ getId(): int
+ getTitle(): String
+ getDate(): String
+ getText(): String
+ setTitle(String): void
+ setDate(String): void
+ setText(String): void

returns
returns
returns
creates
returns

Mission storage and search

SqliteMissionStore
+ DEFAULT_DB_FILE: Path {static}
+ DEFAULT_TSV_FILE: Path {static}
- SELECT: String {static}
- url: String

+ SqliteMissionStore()
+ SqliteMissionStore(Path)
+ getAllMissions(): List<Mission>
+ getMission(int): Mission
+ searchBriefs(String): List<Mission>
+ importFromTsv(Path): int
- query(String, Object): List<Mission>
~ readTsv(Path): List<Mission> {static}

MissionSearchService
- connection: Connection

+ MissionSearchService(Connection)
+ searchBriefs(String): List<Mission>
+ formatSearchResults(String, List<Mission>): String

«interface»
MissionSearch
+ searchBriefs(String): List<Mission>

«interface»
MissionStore
+ getAllMissions(): List<Mission>
+ getMission(int): Mission
+ searchBriefs(String): List<Mission>
+ importFromTsv(Path): int

«interface»
MissionRepository
+ getAllMissions(): List<Mission>

User interface (B)

MissionMenu
- missions: MissionRepository
- search: MissionSearch
- in: Scanner
- out: PrintStream
- audit: AuditLog
- accounts: AccountManager
- user: UserAccount

+ MissionMenu(MissionRepository, MissionSearch, Scanner, PrintStream)
+ MissionMenu(MissionRepository, MissionSearch, Scanner, PrintStream, AuditLog, AccountManager, UserAccount)
+ run(): void
~ showMenu(): void
~ listMissions(): void
~ readMission(): void
~ addUser(): void
- canAddUsers(): boolean
- logEvent(String, String): void
~ searchMissions(): void

Homework5Menu
- database: ModelDatabase
- scanner: Scanner
- out: PrintStream
- audit: AuditLog
- accounts: AccountManager
- user: UserAccount
- admin: boolean

+ Homework5Menu(ModelDatabase, Scanner, PrintStream, AuditLog, AccountManager, UserAccount, boolean)
+ run(): void
- prompt(String): String
- recordsMenu(String): void
- listEntries(String): void
- readEntry(String): void
- addEntry(String): void
- addBrief(): void
- addFacility(): void
- addAgent(): void
- updateEntry(String): void
- updateBrief(int): void
- updateFacility(int): void
- updateAgent(int): void
- auditRead(String): void
- auditCreate(String): void
- auditUpdate(String): void
- deleteEntry(String): void
- manageAccess(): void
- viewAuditLog(): void
- addUser(): void

TopSecret

+ main(String[]): void {static}

creates
creates
creates
creates
creates

Accounts and audit log (D)

«interface»
AuditLog
+ READ: String {static}
+ CREATE: String {static}
+ UPDATE: String {static}
+ DELETE: String {static}

+ record(String, String, String): void
+ getEntries(): List<AuditEntry>

LoginHandler
- accounts: AccountManager
- in: Scanner
- out: PrintStream

+ LoginHandler(AccountManager, Scanner, PrintStream)
+ authenticate(): UserAccount
+ changePassword(): boolean
- createFirstAccount(): void
- setFirstPassword(String): UserAccount
- ask(String): String

SqliteAuditLog
- url: String
- clock: Clock

+ SqliteAuditLog(Path)
+ SqliteAuditLog(Path, Clock)
+ record(String, String, String): void
+ getEntries(): List<AuditEntry>
- isBlank(String): boolean {static}

AccountManager
+ USERNAME_RULE: String {static}
+ PASSWORD_RULE: String {static}
- store: UserStore

+ AccountManager(UserStore)
+ isValidUsername(String): boolean {static}
+ isValidPassword(String): boolean {static}
+ hasAccounts(): boolean
+ getUsers(): List<UserAccount>
+ findUser(String): UserAccount
+ createFirstAccount(String, String): UserAccount
+ addUser(UserAccount, String): UserAccount
+ login(String, String): UserAccount
+ needsPassword(String): boolean
+ setInitialPassword(String, String, String): boolean
+ changePassword(String, String, String, String): boolean
+ setAdmin(UserAccount, String, boolean): void
- savePassword(String, String): boolean
- requireValid(String, String): void {static}

UserAccount
- username: String
- password: String
- admin: boolean

+ UserAccount(String, String, boolean)
+ getUsername(): String
+ getPassword(): String
+ setPassword(String): void
+ isAdmin(): boolean
+ setAdmin(boolean): void
+ hasPassword(): boolean

«interface»
UserStore
+ loadAll(): List<UserAccount>
+ saveAll(List<UserAccount>): void

stores
returns

CipheredUserStore
+ DEFAULT_USERS_FILE: Path {static}
+ DEFAULT_KEY_FILE: Path {static}
~ BUILT_IN_KEY: String {static}
- usersFile: Path
- cipher: Cipher

+ CipheredUserStore()
+ CipheredUserStore(Path, Path)
- readKey(Path): String {static}
+ loadAll(): List<UserAccount>
+ saveAll(List<UserAccount>): void

AuditEntry
- id: int
- timestamp: String
- username: String
- action: String
- item: String

+ AuditEntry(int, String, String, String, String)
+ getId(): int
+ getTimestamp(): String
+ getUsername(): String
+ getAction(): String
+ getItem(): String
+ toString(): String

Homework 3 files and cipher

Cipher
- map: Map<Character, Character>
- reverse: Map<Character, Character>

+ Cipher(String)
+ decipher(String): String
+ encipher(String): String

UserInterface
- programControl: ProgramControl

+ UserInterface(ProgramControl)
+ run(String[]): void
- displayFileList(): void
- displayFileContents(String[]): void

ProgramControl
- fileHandler: FileHandler

+ ProgramControl()
+ getAvailableFiles(): List<String>
+ getFileContents(String): String
+ decipherFile(String): String
+ decipherFile(String, String): String

FileHandler
- dataFolder: Path
- cipherFolder: Path

+ readDataFile(String): String
+ readCipherFile(String): String
+ getAvailableFiles(): List<String>